

Technical Writeup: GALLa Stage 1

Graph Encoder Pretraining

Reference: Zhang et al., GALLa: Graph Aligned Large Language Models for Improved Source Code Understanding, arXiv:2409.04183v4, Sep 2025

Prepared for research discussion — IR-to-Embeddings / Graph-based Code Representation

1. Context and Motivation

GALLa (Graph Aligned Large Language Models) addresses a fundamental tension in code understanding: large language models process source code as flat token sequences and miss the rich structural semantics captured by program graphs (AST, DFG), while models that do encode graph structure require architectural modifications incompatible with pretrained decoder-only LLMs.

The core idea is borrowed from vision-language alignment (LLaVA, Qwen-VL): use a lightweight adapter to project graph encoder outputs into the LLM's embedding space, treating graphs as a second modality. This keeps the LLM architecture untouched and makes the framework both model-agnostic and task-agnostic.

GALLa's training is split into two stages. This writeup focuses exclusively on **Stage 1: Graph Encoder Pretraining** — the self-supervised phase where the GNN and adapter learn to produce graph tokens that the frozen LLM can decode into source code.

2. Architecture of the Pretraining Stage

The Stage 1 model comprises three modules connected in a pipeline. Only the first two are trainable; the LLM is frozen throughout.

2.1 GNN Encoder (Trainable)

Input. A program graph $G = (V, E)$ — either an AST or a DFG — extracted from source code. Each node corresponds to a code snippet (e.g., a variable, an expression, a statement). The node feature matrix $V \in \mathbb{R}^{n_v \times d_{\text{node}}}$ is constructed by encoding each node's source text with a frozen text encoder (e.g., CodeT5+). The edge matrix $E \in \mathbb{R}^{n_e \times 2}$ stores directed edge indices.

Processing. A directed graph neural network processes (V, E) to produce contextual node representations $H \in \mathbb{R}^{n_v \times d_{\text{gnn}}}$. The authors use **DUPLEX** (Ke et al., ICML 2024) — a dual GAT architecture for directed graphs — with 1024 hidden dimensions and ~7M parameters. However, any convolution-based or attention-based directed GNN (e.g., MagNet) can be substituted, since code graphs like DFG are inherently directed.

Why a GNN and not linearization? Prior approaches (SPT-Code, UniXcoder) linearize ASTs via DFS traversal. This works for trees but fails for DFGs, which contain cycles and complex edge patterns. A GNN natively handles loops, node degrees, and edge directionality — all of which carry semantic information about data flow.

2.2 Cross-Attention Adapter (Trainable)

The adapter bridges the GNN's output space (d_{gnn}) to the LLM's embedding space (d_{lm}). Following Qwen-VL, the adapter is a **single cross-attention layer** with the following formulation:

$$X_g = \text{CrossAttn}(q = Q, k = H, v = H)$$

where $Q \in \mathbb{R}^{n_g \times d_{\text{lm}}}$ are n_g **learnable query vectors** (randomly initialized), and H is the GNN's output serving as both keys and values.

The output $X_g \in \mathbb{R}^{n_g \times d_{\text{lm}}}$ consists of n_g embedding vectors in the LLM's space — these are called **"graph tokens"**.

Key design choice. Cross-attention (vs. MLP) allows information exchange between graph nodes during projection. However, ablation studies (Table 6 in the paper) show that a 3-layer MLP adapter performs comparably or even slightly better — possibly because the MLP has more layers (3 vs. 1) and thus stronger expressivity per parameter.

2.3 Frozen LLM Decoder

The graph tokens X_g are concatenated with any text token embeddings X_t (obtained from the LLM's own embedding layer) to form the combined input:

$$X = [X_g, X_t] \in \mathbb{R}^{(n_g + n_t) \times d_{\text{lm}}}$$

This is fed into the LLM's Transformer layers. The LLM produces output logits Y via standard causal language modeling. Crucially, the cross-entropy loss is **masked on graph token positions** and computed only on text token positions — the model is never asked to predict graph tokens, only to use them as context for generating text.

LLM weights are entirely frozen in Stage 1. The rationale: the GNN and adapter are randomly initialized. If the LLM were trainable, noisy gradients from these untrained components would corrupt the LLM's pretrained representations. By freezing the LLM, it serves as a stable decoding target — the GNN+adapter must learn to produce embeddings that the LLM already knows how to decode.

3. Training Objective: Graph2Code

Stage 1 uses a single self-supervised task: **Graph-to-Code generation (Graph2Code)**. Given graph tokens X_g derived from a program's AST/DFG, the model must reconstruct the corresponding source code tokens X_t by maximizing:

$$P(X_t \mid X_g)$$

This is standard autoregressive next-token prediction (causal LM loss), conditioned on graph tokens instead of (or in addition to) text tokens. The loss formulation is cross-entropy, masked to exclude graph token positions.

Intuition. The authors draw an explicit analogy to LLaVA's Stage 1, where a frozen LLM learns to decode image features projected by a vision encoder + linear layer. Here, the GNN + cross-attention adapter are the "graph tokenizer" — learning to convert structural graph representations into a sequence of embedding vectors that the frozen LLM can understand and decode back into source code.

What the model must learn. For the frozen LLM to successfully generate source code from graph tokens alone, the GNN+adapter must encode sufficient information about: (a) the syntactic structure (from AST — nesting, parent-child relationships), (b) the data flow semantics (from DFG — variable use-define chains, data dependencies), and (c) the lexical content of each node (variable names, operators, literals). The adapter's learned queries must attend to and compress this into n_g dense vectors aligned with the LLM's embedding space.

4. Data Pipeline

4.1 Graph Extraction

Source programs are drawn from CodeNet (Puri et al., 2021): ~240K Python programs and ~75K Java programs. For each program, one AST and one DFG are extracted using the JetBrains Program Structure Interface (PSI), yielding 626K code-graph pairs after filtering out empty graphs and source files exceeding 4096 tokens.

A key detail: the authors use Universal AST (UAST), where node types are abstracted to be language-independent (e.g., "variable", "binary expression", "return statement" rather than Python-specific or Java-specific types). There are 43 node types total. This design choice supports cross-language generalization — a property validated in the experiments where Python+Java alignment improves performance on JavaScript and C tasks.

4.2 Graph2Code Data for Stage 1

For the Graph2Code task specifically, Python programs without a detectable main function are removed, resulting in ~150K Java and ~81K Python graph-code pairs. Each sample consists of: (a) the graph (nodes + edges) as input to the GNN, and (b) the corresponding source code text as the generation target for the LLM.

5. Training Configuration

Parameter	Value
Trainable modules	GNN (DUPLEX, 7M params) + Cross-Attention Adapter

Frozen modules	LLM (all weights), Text encoder (CodeT5+)
GNN hidden dimension	1024
Epochs	15
Learning rate	1e-4
Warmup steps	1,000
Weight decay	0.1
Global batch size	240
Optimizer	AdamW
Parallelism	ZeRO Stage 2
Hardware	2 machines x 8 A100 80G GPUs
Graph data source	CodeNet (240K Python + 75K Java programs)
Final graph-code pairs	~231K (150K Java + 81K Python)

Table 1: Stage 1 training hyperparameters and setup.

6. End-to-End Data Flow (Single Training Step)

The following traces the complete forward pass for one Graph2Code training sample:

Step 1: Graph Extraction. Source code $\rightarrow$ parser (PSI) $\rightarrow$ AST and DFG. Each node maps to a code snippet; edges capture parent-child (AST) or use-define (DFG) relationships.

Step 2: Node Feature Extraction. A frozen text encoder (CodeT5+) embeds each node's source text into a fixed-size vector. This produces $V \in \mathbb{R}^{n_v \times d_{\text{node}}}$.

Step 3: GNN Forward Pass. DUPLEX processes (V, E) through message passing layers, producing contextual node representations $H \in \mathbb{R}^{n_v \times d_{\text{gnn}}}$ that incorporate neighborhood information, edge direction, and graph topology.

Step 4: Adapter Projection. The cross-attention adapter uses n_g learnable queries Q to attend over H , producing graph tokens $X_g \in \mathbb{R}^{n_g \times d_{\text{lm}}}$. This is a dimensionality-changing and information-compressing step.

Step 5: LLM Input Assembly. Graph tokens X_g are concatenated with any text prompt embeddings X_t to form $X = [X_g, X_t]$.

Step 6: Autoregressive Decoding + Loss. The frozen LLM processes X via causal attention and produces logits. Cross-entropy loss is computed only on text token positions (graph token positions are masked). Gradients flow back through the LLM (without updating it) to the adapter and GNN.

7. Key Design Rationales and Constraints

7.1 Why Freeze the LLM?

The GNN and adapter are randomly initialized. Training them jointly with the LLM from the start would produce large, noisy gradients that degrade the LLM's pretrained representations. By freezing the LLM, Stage 1 reduces to learning a projection: "what sequence of embedding vectors, when fed to this specific frozen LLM, will cause it to generate this specific source code?" This is a well-conditioned optimization problem because the LLM provides a stable, informative gradient signal.

7.2 Why Graph2Code (and not other objectives)?

Graph2Code is a reconstruction objective — the model must recover the original source from its graph representation. This forces the graph tokens to be informationally complete: they must capture enough about the program's structure, semantics, and lexical content for the LLM to regenerate it. Any information lost in the GNN → adapter pipeline will manifest as reconstruction errors, providing a direct training signal.

More complex objectives like GraphQA (edge prediction, parent/child prediction) are reserved for Stage 2, where the LLM is unfrozen and can learn to reason about graph structures. In Stage 1, the simpler reconstruction task is sufficient to establish a working alignment between graph space and LLM embedding space.

7.3 Loss Masking on Graph Tokens

The loss is computed only on text token positions. This means the model is never asked to "predict" graph tokens — it only uses them as conditioning context. This is consistent with how VLMs handle image tokens: they appear in the input but are not prediction targets. The graph tokens serve purely as an information bottleneck through which structural knowledge must pass.

7.4 Separation of Alignment Data from Task Data

A critical design choice: the graph alignment data (CodeNet programs) is entirely separate from any downstream task data. This means Stage 1 does not require graph annotations for task-specific datasets — a major practical advantage since extracting ASTs/DFGs requires syntactically correct, complete code, which many downstream tasks (defect detection, code repair) do not guarantee.

8. Relevant Ablation Findings

Several ablation results from the paper are directly relevant to understanding what Stage 1 learns:

DFG > AST for alignment. When using only AST or only DFG for Graph2Code, DFG produces larger improvements (Table 5). The authors attribute this to AST being closely related to the surface form of code (and thus redundant with what the LLM already knows from text), while DFG encodes more novel structural information like loops and data dependencies.

Graph information matters (not just extra compute). A control experiment (Figure 3, Table 5) trains the LLM on the same source code from the alignment corpus but without any graph input — essentially continual pretraining. This performs worse than Graph2Code with actual graph tokens, confirming that the structural graph information itself is the source of improvement, not just additional training data.

Cross-language generalization. Although graph alignment uses only Python and Java programs, it improves performance on JavaScript summarization and C defect detection. This suggests that the structural knowledge learned in Stage 1 (and refined in Stage 2) is language-agnostic — supporting the use of Universal AST.

Module interchangeability. Replacing DUPLEX with MagNet (convolution-based GNN) or replacing cross-attention with MLP produces comparable results (Table 6), confirming that GALLa is a framework, not tied to specific component choices.

9. Connections to IR-to-Embeddings Research

The following observations may be relevant when thinking about how GALLa's pretraining paradigm connects to IR-based code representation research (e.g., IR2Vec, LLVM IR embeddings, Graph Transformers over program graphs):

1. The "align then discard" pattern. GALLa's most distinctive feature is that the GNN and adapter are used only during training and discarded at inference. The structural knowledge is transferred into the LLM's weights (in Stage 2) as an implicit inductive bias. This pattern could be applied to IR embeddings: use IR-derived graph representations to align/regularize a model during training, then discard the IR pipeline at inference time.

2. Graph tokens as a compression bottleneck. The adapter compresses variable-length graph representations (n_v nodes) into fixed-length n_g graph tokens. This is analogous to producing a fixed-size embedding from an IR graph — the cross-attention mechanism with learnable queries is a differentiable pooling strategy that could replace or augment approaches like TransE for generating program-level embeddings from graph structures.

3. Reconstruction as a pretraining signal. Graph2Code forces the embedding to be informationally complete with respect to the source code. For IR embeddings, an analogous objective could be: given a Graph Transformer's encoding of a program's IR graph, reconstruct the original IR (or source code). This would ensure the embedding retains the semantic content needed for downstream tasks.

4. Universal graph abstractions. The UAST design — 43 language-independent node types — parallels the role of LLVM IR as a language-independent intermediate representation. Both serve as canonical forms that abstract away surface-level language differences. GALLa's success with UAST suggests that operating on a universal IR (like LLVM IR) for graph-based alignment could yield similar cross-language transfer.

5. Feasibility on limited compute. Stage 1 trains only ~7M GNN params + a small adapter, with the LLM frozen. This is computationally light compared to full LLM pretraining. A similar controlled

ablation — e.g., replacing TransE with a Graph Transformer while keeping everything else fixed — is feasible on Colab-scale resources, as the trainable parameter count remains small.

10. Summary

GALLa's Stage 1 pretraining is a self-supervised alignment phase that trains a GNN encoder and cross-attention adapter to project program graph representations into a frozen LLM's embedding space. The Graph2Code reconstruction objective ensures informationally rich graph tokens. The design is modular (any GNN, any adapter, any LLM), computationally efficient (only ~7M+ trainable params), and produces a "graph tokenizer" that, once the LLM is unfrozen in Stage 2, enables the LLM to internalize structural knowledge about code — knowledge it then retains even after the graph components are discarded at inference time.